AI

AI-Powered Sales Analyst

Project Report

June 2026

1. Executive Summary

AI-Powered Sales Analyst is a full-stack, AI-powered business intelligence web application built with Python and Streamlit. It enables business users to upload their sales CSV data and immediately receive interactive dashboards, machine-learning-driven sales forecasts, and natural-language insights powered by the DeepSeek LLM API — all through a polished, production-ready UI.

The application targets sales managers, analysts, and small-to-medium businesses that need quick, actionable insights without the overhead of heavyweight BI platforms. No coding knowledge is required; the entire experience is point-and-click.

2. Project Overview

Attribute	Details
Project Name	AI-Powered Sales Analyst
Application Type	Web Application (Streamlit)
Primary Language	Python 3
AI Integration	DeepSeek LLM (deepseek-chat model)
ML Framework	scikit-learn (RandomForestRegressor)
Entry Point	app.py
Data Format	CSV (date, product, category, quantity, unit_price, discount, region)
Deployment	streamlit run app.py

3. Project Objectives

- Provide an interactive sales dashboard with real-time KPIs, trend charts, and product breakdowns.
- Enable ML-powered sales forecasting from user-supplied inputs (quantity, price, discount).
- Offer a natural-language AI chat interface for querying sales data without writing code.
- Deliver a professional, themeable UI with minimal setup requirements.
- Support both sample data (bundled CSV) and live user-uploaded data.

4. System Architecture

The application follows a modular layered architecture separating concerns cleanly across four layers:

Layer	Files	Responsibility
Presentation	app.py, utils/theme.py, utils/ui_components.py	Page routing, global CSS theme, branded sidebar, KPI cards, insight cards
Visualization	utils/charts.py	Matplotlib line, bar, and pie charts rendered via st.pyplot()
Machine Learning	ml/train_model.py, ml/predict.py, models/sales_model.pkl	RandomForest model training, serialization with joblib, and inference
AI / NLP Service	services/deepseek.py	REST integration with DeepSeek Chat API for data-aware Q&A
Data Layer	utils/preprocess.py, data/sample.csv	CSV ingestion, total_sales feature derivation, category filtering
Configuration	config.py, .env	Environment variable loading (API key, token limits)

5. Key Features

5.1 Dashboard Page

- Two gradient KPI cards: Total Sales (sum) and Top Product (by revenue).
- Sales Trend line chart grouped by date.

- Product Sales bar chart and Product Distribution pie chart rendered side-by-side.
- Three auto-generated AI business insight cards (Revenue Leader, Growth Opportunity, Needs Attention) derived from live data.
- Sidebar category filter (selectbox) scoped to whichever category the user chooses.

5.2 Prediction Engine Page

- User inputs: Quantity, Unit Price, Discount (%).
- Calls the pre-trained RandomForestRegressor to predict total_sales.
- Displays predicted value with a Streamlit bar chart for visual confirmation.
- Gracefully degrades with an actionable error if the model file is missing.

5.3 AI Chat Page

- Free-text question input sent to DeepSeek alongside the full DataFrame as context.
- Answers rendered in a styled white card with subtle shadow.
- Token budget controlled via MAX_TOKENS_PER_PROMPT environment variable (default 30).
- Clear error messaging for missing API key or network failures.

5.4 Data Handling

- Sidebar CSV uploader (drag-and-drop) with st.cache_resource for model persistence.
- One-click 'Load sample data' button for instant demo mode.
- clean_data() derives total_sales = quantity × unit_price × (1 − discount/100).
- Session state preserves the loaded DataFrame across page navigations.

6. Technology Stack

Component	Technology	Version / Notes
Web Framework	Streamlit	Latest stable
Data Processing	Pandas	DataFrame ingestion and manipulation
Numerical Ops	NumPy	Underpins pandas and sklearn
ML Model	scikit-learn	RandomForestRegressor
Model Serialization	joblib	Fast pkl serialization
Visualization	Matplotlib / Seaborn	st.pyplot() charts
AI Chat	DeepSeek API	deepseek-chat model via REST

Component	Technology	Version / Notes
HTTP Client	requests	DeepSeek API calls
Config	python-dotenv	.env file loading
Fonts / CSS	DM Sans (Google Fonts)	Injected via st.markdown()

7. Data Model

The application expects CSV files with the following schema:

Column	Type	Description
date	string / date	Transaction date (YYYY-MM-DD)
product	string	Product name
category	string	Product category (used for sidebar filter)
quantity	integer	Units sold in the transaction
unit_price	float	Price per unit (in local currency)
discount	float	Discount percentage applied (0–100)
region	string	Geographic region of the sale
total_sales*	float	Derived: quantity × unit_price × (1 − discount/100)

**total_sales is not present in the raw CSV; it is computed by clean_data() on every upload.*

8. Machine Learning Model

8.1 Algorithm

The project uses a Random Forest Regressor from scikit-learn. Random Forests are ensemble learners that average predictions from many decision trees, offering robustness to overfitting and good performance on tabular data with minimal hyperparameter tuning.

8.2 Training Pipeline

- Load data/sample.csv and derive total_sales.
- Select features: quantity, unit_price, discount.
- Split: 70% training / 30% test (random split, default seed).

- Fit RandomForestRegressor with default hyperparameters.
- Serialize model to models/sales_model.pkl via joblib.

8.3 Inference

- At startup, app.py loads the pkl file into a cached resource.
- Prediction page calls model.predict([[quantity, unit_price, discount]])[0].
- The predict.py helper exposes a predict_sales(data) function for programmatic use.

9. AI Chat Integration

The DeepSeek integration (services/deepseek.py) sends two messages to the deepseek-chat model:

- System prompt: instructs the model to act as a sales analyst and constrains response length to MAX_TOKENS_PER_PROMPT tokens.
- User message: concatenates the DataFrame as a string with the user's question.

Configuration is handled via .env:

Variable	Default	Purpose
DEEPSEEK_API_KEY	(required)	Bearer token for DeepSeek API
MAX_TOKENS_PER_PROMPT	30	Max response tokens (controls cost & speed)

Error handling covers: missing API key, HTTP request failures (timeout=60s), and unexpected JSON shapes.

10. UI / UX Design

The theme is defined entirely in utils/theme.py and injected once via apply_theme(). Key design decisions:

Element	Design Choice
Font	DM Sans (Google Fonts) — modern, readable
Main Background	Soft gradient: slate-50 → indigo-50 → slate-100
Sidebar	Dark navy gradient (#0f172a → #1e293b) for contrast
Primary Color	Indigo (#4F46E5 / #6366f1) used for buttons, accents, table headers

Element	Design Choice
KPI Cards	Gradient backgrounds (purple-blue, red-pink) with white text
Insight Cards	Color-coded (green=success, blue=info, amber=warning) with left border accent
Charts	White panel with rounded corners and drop shadow
Empty State	Dashed border with centered icon and instructional copy
Buttons	Gradient indigo with hover lift animation

11. Setup & Running the Application

11.1 Prerequisites

- Python 3.9 or higher
- pip package manager
- A DeepSeek API key (for AI Chat feature)

11.2 Installation Steps

1. Clone or extract the project folder.
2. `Install dependencies: pip install -r requirements.txt`
3. `Create a .env file: DEEPSEEK_API_KEY=your_key_here`
4. `Train the ML model: python ml/train_model.py`
5. `Launch the app: streamlit run app.py`

12. Conclusion

AI-Powered Sales Analyst is a well-structured, visually polished application that successfully combines three distinct technologies — Streamlit UI, scikit-learn ML, and the DeepSeek LLM API — into a cohesive product. The modular codebase makes it straightforward to extend with new chart types, additional ML models, or alternative AI providers.

The application delivers real business value by enabling non-technical users to explore sales data, forecast revenue, and ask natural-language questions — all through a single, intuitive interface.